GenSpark

Linux/CLI; Docker basics; CI/CD Introduction

As an engineer, especially in a role where you're often setting up your own environment and troubleshooting deployments without a large infrastructure team behind you, comfort with the command line, containers, and automated pipelines is foundational. This article introduces all three at a working level.

Linux and the Command Line

Most servers, cloud environments, and development tools run on Linux, and most professional engineering work happens through the command line (also called the CLI, or terminal) rather than clicking through a graphical interface. Learning to navigate comfortably here isn't optional — it's how you'll interact with almost every tool covered in this program.

A few categories of commands you'll use constantly:

Navigation and file management:

```
pwd          # show current directory
ls           # list files in current directory
cd folder/   # move into a directory
mkdir name   # create a new directory
rm file.txt  # remove a file
```

Viewing and editing files:

```
cat file.txt     # print file contents
nano file.txt    # open a simple text editor
grep "text" file # search for a string inside a file
```

Permissions and processes:

```
chmod +x script.sh   # make a file executable
ps aux               # list running processes
kill 1234            # stop a process by its ID
```

You don't need to memorize every command up front. What matters is building the habit of reaching for the terminal instead of avoiding it, and knowing how to look up a command's options when you need them (`man command` or `command --help`).

Docker Basics

The problem Docker solves: code that runs fine on your machine sometimes fails on someone else's, or in production, because of differences in installed software versions, operating systems, or configuration. Docker solves this by packaging an application together with everything it needs to run — code, runtime, libraries, settings — into a single unit called a **container**.

A few core concepts:

- **Image**: a blueprint for a container. It defines what's installed and how the application is configured. Images are built from instructions in a file called a `Dockerfile` .
- **Container**: a running instance of an image. You can start, stop, and remove containers without affecting the underlying image.
- **Dockerfile**: a text file with step-by-step instructions for building an image — what base operating system to start from, what dependencies to install, what command to run when the container starts.

A minimal Dockerfile for a Python app might look like:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Basic commands you'll use:

```
docker build -t myapp .     # build an image from a Dockerfile
docker run myapp            # run a container from that image
docker ps                   # list running containers
docker stop <container_id>  # stop a running container
```

The key benefit: a containerized application behaves the same way on your laptop, in a teammate's environment, and in production, because the container carries its own self-contained environment with it.

CI/CD Fundamentals

CI/CD stands for **Continuous Integration** and **Continuous Delivery/Deployment**. It's the automated process that takes code from a developer's pushed commit through testing and, often, all the way to deployment — without someone manually running each step by hand.

Continuous Integration (CI) means that every time code is pushed, an automated process runs to check it — usually running the test suite, and often checks like linting (style/formatting checks) or security scans. The goal is to catch problems immediately, while the change is small and fresh in the engineer's mind, rather than discovering them later when they're tangled up with other changes.

Continuous Delivery/Deployment (CD) extends this further: once code passes CI, it's automatically packaged and, depending on the setup, either prepared for deployment (delivery) or actually deployed to a live environment (deployment) without manual intervention.

A typical CI/CD pipeline for a simple application might run these stages, in order, every time code is pushed:

1. **Build** — install dependencies, compile if necessary
2. **Test** — run the automated test suite
3. **Lint/Static Analysis** — check code style and catch common errors
4. **Package** — build a deployable artifact (often a Docker image)
5. **Deploy** — push the artifact to a server or environment

If any stage fails, the pipeline stops and reports the failure, rather than allowing broken code to move forward.

Why This Matters for an FDE

You'll frequently be responsible for setting up your own environment, containerizing an application so it runs consistently for a client, and working with (or building) pipelines that automatically test and deploy your code. Being comfortable at the command line, understanding what a container actually does, and knowing how a CI/CD pipeline is structured means you can troubleshoot problems yourself instead of being stuck when something doesn't "just work."

Comprehension Check

1. Why does Docker solve the "works on my machine" problem?
2. What's the difference between Continuous Integration and Continuous Deployment?
3. If a CI/CD pipeline fails at the "Test" stage, what does that tell you, and what should happen next?

← Previous: Git workflows and PR hygiene

Next: Python Foundations: Virtual Environments, History, and the Basics →